Lab 3: News Categorization using TF-IDF and Naïve Bayes

Objective

The objective of this lab is to implement a News Categorization system using Machine Learning. Students will learn how to preprocess textual data, convert text into numerical features using TF-IDF, train a Naïve Bayes classifier, and evaluate its performance on a news dataset.

Problem Statement

News categorization (also known as news classification) is the task of automatically assigning news articles to predefined categories such as:

- Business
- Entertainment
- Politics
- Sport
- Technology

In this lab, you will build a machine learning model that classifies news articles into their respective categories using TF-IDF feature extraction and the Multinomial Naïve Bayes algorithm.

Term Frequency (TF)

Term Frequency measures how frequently a word appears in a document.

$$TF(t,d) = \frac{\text{Number of occurrences of term } t}{\text{Total number of terms in document } d}$$

Inverse Document Frequency (IDF)

Inverse Document Frequency measures the importance of a word across all documents.

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t}\right)$$

TF-IDF

TF-IDF combines TF and IDF to determine the importance of a word in a document relative to the entire corpus.

$$TFIDF(t,d) = TF(t,d) \times IDF(t)$$

Words that occur frequently in a document but rarely across the corpus receive higher TF-IDF scores.

Dataset

The BBC News Dataset contains news articles categorized into multiple topics.

Features

Feature	Description
ArticleID	Article ID
Text	News article content
Category	News category label

Procedure

Step 1: Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

```
df = pd.read_csv('bbc_news_dataset.csv')
print(df.info())
```

Step 3: Separate Features and Labels

```
X = df['Text']
y = df['Category']
```

Step 4: Explore Categories

```
category_names = df['Category'].unique()
print(category_names)

value_counts = df['Category'].value_counts()
print(value_counts)
```

Step 5: Visualize Category Distribution

```
plt.bar(x=category_names, height=value_counts)
plt.title('Categories Distribution')
plt.xlabel('Category')
plt.ylabel('Frequency')
plt.show()
```

Step 6: Generate Word Clouds

```
from wordcloud import WordCloud

for category in category_names:
    text = " ".join(df[df['Category']==category]['Text'].values)
    wc = WordCloud(width=800,height=400,
                   background_color='white').generate(text)
    plt.figure(figsize=(8,4))
    plt.imshow(wc)
    plt.axis('off')
    plt.show()
```

Step 7: Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 8: Build Model

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

model = make_pipeline(
    TfidfVectorizer(stop_words='english'),
    MultinomialNB()
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Step 9: Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Step 10: Confusion Matrix

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm,
display_labels=category_names).plot()
```

Step 11: Test with New Articles

```
texts = ['Enter any news article text here.']
prediction = model.predict(texts)
print(prediction)
```

Step 12: Save Model

```
import joblib

joblib.dump(model,'model.joblib')
print('SUCCESS')
```

Tasks

1. Evaluate the model using Accuracy, Precision, Recall, and F1-Score. Analyze the results.
2. Experiment with TF-IDF parameters such as max_features, ngram_range, and min_df.
3. Classify at least five custom news articles and record the predicted categories.
4. Save the trained model and reload it to perform predictions on new data
5. Experiment with Train-Test Split Ratio
6. Evaluate the model using different train-test splits:
   70% – 30%
   80% – 20%
   90% – 10%
   Compare the resulting accuracies.
7. Perform 5-Fold Cross Validation and calculate Mean Accuracy and Standard Deviation .
8. Discuss whether the model is stable across different folds.
9. Train and prepare a comparison table showing: Accuracy , Training Time , Prediction Time.
   - Multinomial Naïve Bayes
   - Logistic Regression
   - Support Vector Machine (SVM)
10. Build a News Recommendation Prototype: After classifying an article, display three similar articles from the same category using cosine similarity.
11. Deploy the trained model using a simple web interface with either Flask or Streamlit and allow users to enter a news article and obtain the predicted category in real time.